

Первые программы

Ввод, числа, условия, циклы и строки. И две первые настоящие игры, которые ты напишешь сам.

Четверть 1 · 7 уроков

ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 1.** Разговор с программой — input
2. **Урок 2.** Числа и калькулятор
3. **Урок 3.** Условия: программа решает
4. **Урок 4.** 🎮 Проект «Угадай число»
5. **Урок 5.** Циклы: повторяем действия
6. **Урок 6.** Строки: работаем с текстом
7. **Урок 7.** 🎮 Проект «Виселица»

Разговор с программой: команда `input`

ЧЕМУ НАУЧИМСЯ

- Спрашивать у пользователя данные командой `input`
- Сохранять ответ в переменную и использовать его
- Написать программу, которая знакомится с человеком



Программа умеет спрашивать

Раньше все данные мы записывали в код сами. Но настоящая программа **общается**: спрашивает у человека и получает ответ. За это отвечает команда `input` — «спроси и подожди ответ».



На пальцах. `input` — это как продавец, который спрашивает «что вам?» и ждёт. Пока ты не ответишь, программа стоит на месте. Ответ она кладёт в ящик-переменную, чтобы использовать дальше.



Разбираем код

Программа спрашивает имя и город, потом здоровается:

znakomstvo.py

```
name = input("Как тебя зовут? ")
city = input("Из какого ты города? ")

print("Привет, " + name + "!")
print("Значит, ты из города " + city + ". Отличное место!")
```

ПРИМЕР РАБОТЫ

```
Как тебя зовут? Алишер
Из какого ты города? Худжанд
Привет, Алишер!
Значит, ты из города Худжанд. Отличное место!
```

Знак `+` здесь **склеивает** текст: строку «Привет, », ответ из ящика `name` и «!». Это называется соединением строк.

ВАЖНО ЗАПОМНИТЬ

input **всегда возвращает текст**, даже если ввели число. «15» после input — это текст, а не число. Как превратить его в число — узнаем на следующем уроке.

👉 ЗАДАНИЕ В ПАРЕ

Анкета друга

Программа спрашивает имя, любимую еду и любимую игру, а потом составляет фразу: « **имя** любит **еда** и играет в **игра** ».

Водитель печатает, штурман диктует. Через 10 минут — меняетесь.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
name = input(Как зовут?)
```

```
print("Привет " name)
```

А ТАК ПРАВИЛЬНО

```
name = input("Как зовут? ")
```

```
print("Привет " + name)
```

Вопрос внутри input тоже в кавычках. А чтобы приклеить переменную к тексту — нужен знак `+`.



ДОМАШНЕЕ ЗАДАНИЕ

1. Программа спрашивает имя и класс, затем печатает: « **имя** учится в **класс** классе ».
2. Программа спрашивает город и отвечает: «Я бы хотел побывать в **город** !»

★ **Со звёздочкой:** спроси имя друга и «нарисуй» ему открытку из символов с его именем внутри.

Числа и калькулятор

ЧЕМУ НАУЧИМСЯ

- Превращать текст в число командой `int`
- Считать: сложение, вычитание, умножение, деление, остаток
- Написать калькулятор из двух чисел



Почему `input` мало

На прошлом уроке мы узнали: `input` даёт **текст**. С текстом нельзя считать — компьютер не станет складывать «2» и «3» как числа, он их склеит в «23». Чтобы считать, текст нужно превратить в число командой `int`.



На пальцах. «5» в кавычках — это как цифра, нарисованная на бумаге: красивая, но сложить нельзя. `int` превращает нарисованную цифру в настоящее число, с которым уже можно работать.



Разбираем код

Калькулятор: спрашиваем два числа и показываем все действия:

```
calc.py

a = int(input("Введи первое число: "))
b = int(input("Введи второе число: "))

print("Сумма:", a + b)
print("Разность:", a - b)
print("Произведение:", a * b)
print("Деление:", a / b)
print("Остаток:", a % b)
```

ЕСЛИ ВВЕСТИ 10 И 3

Сумма: 13

Разность: 7

Произведение: 30

Деление: 3.33...

Остаток: 1

Обрати внимание: `int(input(...))` — это две команды сразу. Сначала `input` спрашивает, потом `int` превращает ответ в число. А запятая в `print` ставит пробел между текстом и результатом.

ЗНАКИ ДЕЙСТВИЙ В PYTHON

`+` сложить · `-` вычесть · `*` умножить · `/` разделить · `%` остаток от деления. Остаток очень пригодится дальше — он умеет проверять чётность.

👉 ЗАДАНИЕ В ПАРЕ

Магазинный чек

Программа спрашивает цену товара и количество, потом считает и выводит общую сумму в сомони. Подсказка: не забудь `int` для обоих ответов.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
a = input("Число: ")
print(a + 5)
```

```
print("Сумма:" a + b)
```

А ТАК ПРАВИЛЬНО

```
a = int(input("Число: "))
print(a + 5)
```

```
print("Сумма:", a + b)
```

Без `int` ответ остаётся текстом. И между кусочками внутри `print` нужна запятая.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Программа спрашивает возраст и считает, сколько тебе будет через 10 лет.
2. Программа переводит сумму в сомони в дирамы (1 сомони = 100 дирам).

★ **Со звёздочкой:** калькулятор чаевых — спроси счёт, добавь 10% и выведи итог.

Условия: программа принимает решения

ЧЕМУ НАУЧИМСЯ

- Учить программу выбирать действие по условию
- Разбирать **if** и **else** — «если ... иначе ...»
- Проверять число на чётность



Что такое условие

Пока наши программы выполняли все команды подряд. Но настоящая программа умеет **выбирать**: сделать одно или другое, смотря по обстоятельствам.



На пальцах. Утром смотришь в окно. **Если** дождь — берёшь зонт. **Иначе** — не берёшь. Это решение по условию. Программа делает так же: проверяет условие и выбирает путь.

В Python это слова **if** (если) и **else** (иначе). После условия — двоеточие, а команды внутри пишутся **с отступом** в 4 пробела. Отступ показывает, что команда относится к условию.



Разбираем код

Программа проверяет возраст:

```
vozrast.py

age = int(input("Сколько тебе лет? "))

if age >= 18:
    print("Ты совершеннолетний.")
else:
    print("Ты ещё несовершеннолетний.")
```

ЕСЛИ ВВЕСТИ 15

Ты ещё несовершеннолетний.

Компьютер проверил: **15 >= 18**? Неправда — значит, он пропустил первую ветку и выполнил ту, что после **else**.

А вот проверка чётности — здесь пригодился остаток **%** с прошлого урока:

```
chet.py
```

```
number = int(input("Введи число: "))
if number % 2 == 0:
    print("Число чётное.")
else:
    print("Число нечётное.")
```

ВАЖНО ЗАПОМНИТЬ

Двоеточие в конце условия. Знак `==` — это «сравнить» (а один `=` — «положить в ящик»). И отступ 4 пробела у команд внутри `if` и `else`.

👉 ЗАДАНИЕ В ПАРЕ

Больше из двух

Программа спрашивает два числа и выводит большее. Подсказка: `if a > b: ... else: ...`

Через 10 минут поменяйтесь ролями.

⚠ Частые ошибки

ТАК НЕ РАБОТАЕТ	А ТАК ПРАВИЛЬНО
<code>if age >= 18</code>	<code>if age >= 18:</code>
<code>if age = 18:</code>	<code>if age == 18:</code>
<code>if age >= 18:</code> <code>print(...)</code>	<code>if age >= 18:</code> <code>print(...)</code>

🏠 ДОМАШНЕЕ ЗАДАНИЕ

- Программа спрашивает оценку (2–5) и пишет словами: «отлично», «хорошо» и т.д. Подсказка: пригодится `elif` — «иначе если».
- Программа-«фейсконтроль»: пускает на фильм с 16 лет, иначе отказывает.

★ **Со звёздочкой:** программа проверяет, високосный ли год (делится на 4).

«Угадай число»

ЧТО ПОСТРОИМ

- Первую настоящую игру: компьютер загадал число, ты угадываешь
- Познакомимся с циклом **while** — «повторяй, пока...»
- Научимся считать попытки и завершать цикл словом **break**

Что за игра

Компьютер загадывает число от 1 до 100. Ты называешь варианты, а он подсказывает: «больше» или «меньше», пока не угадаешь. В конце покажет, за сколько попыток ты справился. Это твоя **первая игра** — и она уместается в 15 строк.



Новое слово — цикл. Цикл **while** означает «повторяй, пока условие верно». Здесь мы повторяем вопрос снова и снова, пока число не угадано. Без цикла пришлось бы копировать один и тот же код сто раз.

Собираем игру

```
guess.py

import random

secret = random.randint(1, 100)
attempts = 0

print("Я загадал число от 1 до 100!")

while True:
    guess = int(input("Твоя догадка: "))
    attempts = attempts + 1

    if guess < secret:
        print("Больше!")
    elif guess > secret:
        print("Меньше!")
    else:
        print("Верно! Попыток:", attempts)
        break
```


- 1 random загадывает число**
`random.randint(1, 100)` — компьютер выбирает случайное число и прячет его в ящик `secret`.
- 2 while True — повторяем без конца**
Цикл крутится, спрашивая догадку снова и снова, пока мы сами его не остановим.
- 3 elif — третья ветка**
Теперь веток три: меньше, больше, точно. Слово `elif` — это «иначе если».
- 4 break останавливает игру**
Когда угадали — `break` выходит из цикла, и игра заканчивается.

ЗАДАНИЕ В ПАРЕ

Соберите и сыграйте

Наберите игру и сыграйте по очереди друг против друга. Работает? Поздравляю — вы написали первую игру!

Один печатает, второй проверяет каждую строку. Потом меняетесь и играете.

ДОМАШНЕЕ ЗАДАНИЕ

- Ограничь число попыток семью. Если не угадал за 7 — программа говорит, какое было число.
- Поменяй диапазон на 1–1000 и посмотри, за сколько попыток реально угадать.

★ **Со звёздочкой:** поменяйтесь ролями с компьютером — теперь ты загадываешь, а программа угадывает.

Циклы for: повторяем по счёту

ЧЕМУ НАУЧИМСЯ

- Повторять действие заданное число раз циклом **for**
- Понимать **range** — «диапазон чисел»
- Напечатать таблицу умножения и обратный отсчёт



Два вида циклов

В «Угадай число» мы видели цикл **while** — «повторяй, пока не угадали». Но часто мы знаем **точно, сколько раз** надо повторить: 10 строк, 5 учеников, 12 месяцев. Для этого удобнее цикл **for**.



На пальцах. **while** — это «мешай суп, пока не будет готов» (не знаешь сколько). **for** — это «отожмись 10 раз» (знаешь точно). Оба повторяют, но по-разному считают.



Разбираем код

Обратный отсчёт и таблица умножения:

```
cikly.py

# обратный отсчёт от 10 до 1
for i in range(10, 0, -1):
    print(i)
print("Старт!")

# таблица умножения на 7
for i in range(1, 11):
    print(7, "x", i, "=", 7 * i)
```

НАЧАЛО ВЫВОДА

```
10
9
8
...
Старт!
7 x 1 = 7
7 x 2 = 14
...
```

Переменная `i` — это счётчик: на каждом повторе она берёт следующее число из диапазона. `range(1, 11)` даёт числа от 1 до 10 (последнее число 11 не входит — важно запомнить!).

КАК ЧИТАТЬ RANGE

`range(1, 11)` — от 1 до 10 (правая граница не входит). `range(10, 0, -1)` — от 10 до 1 в обратную сторону. Третье число — шаг.

👉 ЗАДАНИЕ В ПАРЕ

Сумма от 1 до 100

С помощью цикла `for` посчитайте сумму всех чисел от 1 до 100. Подсказка: заведите ящик `total = 0` и на каждом шаге прибавляйте к нему `i`.

Через 10 минут поменяйтесь ролями.



Частые ошибки

ТАК НЕ РАБОТАЕТ

```
for i in range(1, 10):  
    # ждём до 10
```

```
for i in range(5)
```

А ТАК ПРАВИЛЬНО

```
for i in range(1, 11):  
    # теперь до 10
```

```
for i in range(5):
```

Правая граница `range` не входит. И двоеточие в конце строки `for` — обязательно.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Выведи все чётные числа от 2 до 20.
2. Программа спрашивает число `N` и печатает таблицу умножения на него.

★ **Со звёздочкой:** посчитай факториал числа `N` ($1 \times 2 \times 3 \times \dots \times N$).

Строки: работаем с текстом

ЧЕМУ НАУЧИМСЯ

- Узнавать длину текста и брать отдельные буквы
- Менять регистр: **upper** и **lower**
- Проверять, есть ли буква в слове — это ядро игры «Виселица»



Текст — это тоже данные

Слово в Python называется **строкой**. С ним можно многое: узнать длину, вытащить любую букву, перевести в заглавные, проверить, есть ли в нём нужная буква. Всё это нам понадобится уже на следующем уроке — для игры «Виселица».



На пальцах. Строка — это как бусы из букв. Каждая буква на своём месте, у каждого места есть номер. Только счёт начинается с нуля: первая буква — номер 0.



Разбираем код

```
stroki.py

word = input("Введи слово: ")

print("Длина:", len(word))
print("Первая буква:", word[0])
print("Последняя буква:", word[-1])
print("Заглавными:", word.upper())

letter = input("Какую букву ищем? ")
if letter in word:
    print("Есть такая буква!")
else:
    print("Нет такой буквы.")
```

ЕСЛИ ВВЕСТИ «ПАМИР» И БУКВУ «М»

Длина: 5
Первая буква: п
Последняя буква: р
Заглавными: ПАМИР
Есть такая буква!

`len(word)` — длина, `word[0]` — первая буква (счёт с нуля!), `word[-1]` — последняя. А `letter in word` проверяет, есть ли буква в слове — вот это и есть главная проверка «Виселицы».

ВАЖНО ЗАПОМНИТЬ

Буквы нумеруются с нуля: у слова из 5 букв номера 0, 1, 2, 3, 4. Номер `-1` — это последняя буква. Оператор `in` отвечает «да» или «нет»: есть буква в слове или нет.

👉 ЗАДАНИЕ В ПАРЕ

Разбор имени

Программа спрашивает имя и выводит: его длину, первую и последнюю букву, имя заглавными буквами.

Через 10 минут поменяйтесь ролями.

⚠ Частые ошибки

ТАК НЕ РАБОТАЕТ	А ТАК ПРАВИЛЬНО
<code>word[1]</code> # первая буква	<code>word[0]</code> # первая буква
<code>word.upper</code>	<code>word.upper()</code>

Счёт с нуля: первая буква — `[0]`. И у методов вроде `upper` обязательны скобки в конце.

🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Программа считает, сколько букв «а» в введённом слове.
2. Программа выводит слово наоборот. Подсказка: `word[::-1]`.

★ **Со звёздочкой:** проверь, читается ли слово одинаково слева и справа (палиндром).



«Виселица»

ЧТО ПОСТРОИМ

- Игру, где нужно угадать слово по буквам
- Соберём вместе всё: строки, циклы, условия, списки
- Это итоговый проект четверти — с защитой перед классом



Что за игра

Компьютер загадывает слово и показывает его чёрточками: `_ _ _ _ _`. Игрок называет буквы. Угадал — буква открывается. Ошибся — минус попытка. Слов даётся 6 попыток. Откроешь слово — победа. Это твой **главный проект четверти**.



Собираем игру

```
viselica.py

import random
words = ["худжанд", "душанбе", "памир", "python"]
secret = random.choice(words)

guessed = []
mistakes = 0

while mistakes < 6:
    display = ""
    for letter in secret:
        if letter in guessed:
            display = display + letter + " "
        else:
            display = display + "_ "
    print(display)

    if all(l in guessed for l in secret):
        print("Победа! Слово:", secret)
        break

    guess = input("Назови буквы: ")
    if guess in secret:
        guessed.append(guess)
    else:
```

```
mistakes = mistakes + 1
print("Ошибка:", mistakes, "из 6")
```

1 Список слов и выбор

`words` — это список (несколько слов в квадратных скобках). `random.choice` выбирает одно случайное.

2 Показываем слово чёрточками

Цикл `for` проходит по каждой букве: угадана — показываем её, нет — рисуем `_`.

3 `append` добавляет в список

Угаданную букву кладем в список `guessed` командой `append` — «добавь в конец».

4 Проверка победы

`all(...)` отвечает «да», если все буквы слова уже угаданы. Тогда — победа и `break`.



ИТОГОВЫЙ ПРОЕКТ ЧЕТВЕРТИ

Соберите, доработайте, защитите

Наберите игру в паре. Затем сделайте её своей: добавьте слова про Таджикистан, придумайте поздравление за победу. На защите (2 минуты у проектора) расскажите: что делает игра, как работает, что было сложно.

Оба в паре должны уметь объяснить любую строку кода.



ЧТО ПОКАЗАТЬ НА ЗАЩИТЕ

1. Игра запускается и работает без ошибок.
2. Вы добавили минимум 5 своих слов.
3. Каждый может объяснить, что делает любая строка.

★ **Со звёздочкой:** покажите игроку, сколько попыток осталось, и вставьте поздравление с именем победителя.